编程作业 1: Shell + Network 编程 (共计 15: 4+5+6)

1. 【4 分】通过一个 shell 脚本文件（文件名字为:学号-hw1-q1.sh, 假设你的学号为 SID），且要求随机种子数为学号 SID，然后随机生成 1 组 0 至 2^{32}-1 之间的 131072（=8192*16）个质数，每个质数按行重定向输出到一个文本文件（文件名字为: **SID-hw1-q1-0.txt**，重复上述步骤一共完成 16 组质数文件，每个文件命名方式为 **SID-hw1-q1-x.txt** （其中 **x** = 0,1,...,15）,最后将所有的 16 个 log 文件按照分组号合并输出到一个整体文件（文件名字为: **SID-hw1-q1. txt**）。

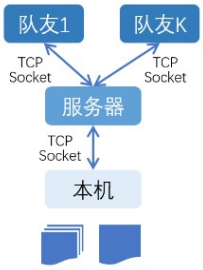
然后该 shell 文件，利用 linux 的 wc 命令统计并打印 16 个分组文件 **SID-hw1-q1-x. txt** 以及合并文件 **SID-hw1-q1.txt** 的 word count 数量，并判断 16 个分组文件的 word count 总数，是否与 **SID-hw1-q1. txt** 的 word count 数量一致。如一致，则打印 YES：the total wc is equal to xxx[即 word count 总数]；否则打印 NO: the total wc of 16 files is XXX[即 16 个分组文件的 word count 总数]，and the wc of the merged file is XXX[即 **SID-hw1-q1. txt** 的 word count 数量]。

2. 【5 分】设计一个 quick sort 的 Python（Java 程序亦可），该程序名为学号**-hw1-q2-qs.py**，对 16 组 **SID-hw1-q1-x.txt** 文件分别按照从小到大的顺序进行排序, 再设计一个 merge sort 的程序(**SID-hw1-q2-ms.py**)对合并文件 **SID-hw1-q1-x.txt** 按照从小到大的顺序进行排序。

然后在本机创建一个脚本文件（文件名字为:**SID-hw1-q2.sh**），要求利用 scp 命令将 16 个分组文件 **SID-hw1-q1-x. txt** 以及 **SID-hw1-q2-qs.py** 排序程序分别上传 upload 至 K 个队友服务器，其中 K=2, 4，则每个队友服务器包含 16/K（=8，4）个分组文件和一个学号**-hw1-q2-qs.py** 排序程序，然后通过 ssh 在每个队友服务器远端执行该脚本，并将排序后的分组文件通过 scp 命令下载并且保存至该生（学号为学号、非队友）本机文件（文件名字为: **SID-hw1-q2-x-qs.txt**），最后将所有 **SID-hw1-q2-x-qs.txt** 文件通过 merge sort 程序(**SID-hw1-q2-ms.py**)合并生成最终从小到大的排序文件学号**-hw1-q2-qs.txt** 文件。统计 upload 分组文件、队友服务器排序该分组文件、download 排序后文件、最终合并排序文件的总体耗时 time-A。在计算耗时 time-A 的过程中，由于有 K 个队友服务器同时并行进行分组文件的 upload、quick_sort、download，因此，仅需要考虑最慢耗时的情况，才计入 time-A 耗时计算。

最后，本机 merge sort 的 Python 程序(**SID-hw1-q2-ms.py**)对合并文件 **SID-hw1-q1. txt** 进行排序并保存至 **SID-hw1-q2-ms.txt** 文件，记录从载入文件 **SID-hw1-q1. txt** 至 **SID-hw1-q2-ms.txt** 文件完成的整体耗时 time-B，计算在不同 K 的情况下的加速比（speedup ratio）time-A/time-B，并讨论如何有效的提高排序加速比

speedup ratio，分析 upload、quick_sort、download 哪个阶段是耗时的瓶颈部分。



3. 【6 分】TCP Socket-based Distributed Merge Sort: 在上述第 2 题的基础上，完成分布式的排序算法，具体要求如图所示，通过增加一个额外的服务器，本机通过把 **SID-hw1-q1-x. txt**、**SID-hw1-q2-qs.py**、**SID-hw1-q1.txt** 和 **SID-hw1-q2-ms.py** 通过 TCP socket 程序上传至服务器，由服务器负责与 K 个队友机器通过 TCP socket 程序进行消息交互，完成第 2 题的内容，即：第 2 题是通过 scp 进行文件传输，第 3 题则由 TCP socket 完成，并对比第 2 题和第 3 题的加速比，各个阶段的延迟对比，讨论第 2 题和第 3 题的实现方案所带来计算的差别和效果。

作业提交要求：

1）deadline：2026-10-18 23:59

2）每题包括 3 类文件：程序文件 PY、排序后结果文件 TXT，运行录屏文件(**doc**)、计算时长和加速比讨论分析一页纸文档 doc，其中执行截屏文件要求截屏记录 console 输入脚本文件并输出结果。要求按照题目创建 3 个子目录（Q1/Q2/Q3）,要求排序结果正确、且缺其中之一目录结构、文件，按比例扣分。

3）在 canvas 上提交最终压缩文件包（**SID-hw1.zip**）